

Should work with any plaintext message and key as input (not hardcoded)

Nigel Boswell

ADD COMMENTS EVERYWHERE

Nigel Boswell

Initial Layout

Create File: Encrypt.asm

Review all lectures and notes

```
1 section .text
2     global _start
3
4 _start:
5     ;Initial Prompt to terminal
6     ;sys_read
7     ;Key Prompt to terminal
8     ;sys_read
9     ;save inputs
10
11 encrypt:
12 decrypt:
13 Create_output_file:
14 Write_output_file:
15
16 exit:
17     mov eax, 1      ;sys_exit
18     mov ebx, 0
19     int 0x80
20
21 section .data
22     ;prompt outputs
23     ;output filename
24 section .bss
25     ;variables
26     ;inputs
27     ;lengths
```

- I will need
- Loops for Encryption and Decryption
 - stdout, sys_read
 - sys_creat, sys_close
 - Jumps for loops iteration and escape/continuation
 - initialized null variables for calculations and result saving
 - Initialized plain text for prompts and result saving/writing to output.file
 - XOR bitwise operation for encrypt/decrypt

- 1) prompt terminal with plaintext word, receive and save input
- 2) prompt terminal with plaintext key, receive and save input
- 3) encryption loop, save final encryption word
- 4) decryption loop, save final decrypted word, should be the same as initial input
- 5) create/rewrite "output.txt" file to display inputs, encryption result, and decryption result
- 6) send these results to terminal
- 7) exit

1 and 2:

a) add prompts to section .data:

```
plain1    db "Plain text: "
plain1_len equ $ - plain1
key1      db "Key: "
key1_len  equ $ - key1
```

b) create code:

;Initial Prompt and read input

```
mov eax, 4      ;sys_write
mov ebx, 1      ;stdout
mov ecx, plain1 ;"Plain text: "
mov edx, plain1_len
int 0x80
mov eax, 3      ;sys_read
mov ebx, 0      ;stdin
mov ecx, msg_buf ;destination for sys_read
mov edx, 128    ;max number of bytes for stdin
int 0x80
```

;This part is important because at the end of the string there WILL be an \n, which will be treated as a character in the word so we gotta get rid of it.

```
dec eax          ;subtract 1 to ignore the \n
mov [msg_len], eax ;saves the true length of the msg into msg_len
```

;Key prompt and read input

```
mov eax, 4      ;sys_write
mov ebx, 1      ;stdout
mov ecx, key1   ;"Key: "
mov edx, key1_len
int 0x80
mov eax, 3      ;sys_read
mov ebx, 0      ;stdin
mov ecx, key_buf ;destination for sys_read
mov edx, 128    ;max number of bytes for stdin
int 0x80
dec eax          ;subtracts 1 to ignore \n
mov [key2_len], eax ;saves the true length of the key into key_len
```

- Encrypt Loop
- a) section .bss -> idx resd 1
Indexing variable for loops
 - b) set idx to 0
 - c) start loop
 - d) compare idx to input length, tells us how far in we are, if done, then skip to next section
 - e) if not done, we need to then account for key inputs which mismatch in length to our plaintext word to encrypt, this is done through using mod.
 $edx = idx \text{ (mod) } key_len$
 - f) perform xor on the current char and the current key char, store result within result variable at current index
 - g) increment idx and loop back

- Decrypt Loop
- a) Set idx to 0, compare idx to msg_len to determine if loop is done.
 - b) suppose not done -> calculate mod same as encryption loop
 - c) perform bitwise xor between curr encrypt char and curr key char
 - d) save result, increment, loop back

Create Output.txt

```
1 write_output_file:
2     mov eax, 8      ;sys_creat
3     mov ebx, filename ;"same as before, set to "output.txt"
4     mov ecx, 0711o  ;permission, all read/write
5     int 0x80
6     mov [fd_out], eax ;save file descriptor
```

```
1 write_file:
2     ;Write "Plain text: " + input word \n
3     ;To output.txt
4     ;"Plain text: "
5     mov eax, 4
6     mov ebx, [fd_out]
7     mov ecx, plain1
8     mov edx, plain1_len
9     int 0x80
```

Basically this over and over but slightly different each time just to add the next line of stuff,

Everything that went wrong:

xor went to chars that were not printable, if key is 0 it would error out, I had several variables too closely named to each other and barely commented to help in bug fixing. and ofc the occasional typo

Fixed after many hours.

This gosh darn video

it took me literally half a day to get it properly done in a "short" enough time jsut for it to be the wrong file format, have no audio, be the wrong size. I ended up downloading about 3 applications and having to learn them to do this video. I then had to create a youtube channel for creators and every name I pciked was taken and it cancelled me out because it thought I was a bot. I then had to log into my burner and try again. But golly man I did it